

Leopard: Hardware Pass-Through Remote Storage Access with Queue Concurrency for Edge Intelligent Workstations

Wenjie Wang, Bo Peng, Jianguo Yao, Haibing Guan

Shanghai Jiao Tong University, Shanghai, China

{wang_wen_jie, pengbo_michael, jianguo.yao, hbguan}@sjtu.edu.cn

Abstract—Edge intelligent workstations load (store) massive empirical data from (to) remote cloud storage due to limited local storage. However, current remote storage access frameworks are complex. They use expensive computing resources to manipulate multiple concurrent request queues in modern high-speed storage devices for saturating performance. Complex software stacks and limited CPUs on edge intelligent workstations hinder saturating concurrent request queues, thus resulting in up to 75% performance degradation for remote storage. We propose Leopard, a hardware pass-through remote storage access framework with queue concurrency, which provides lossless remote storage access for edge intelligence. Leopard proposes a custom NVMe controller using SmartNIC's FPGA core to emulate it as an NVMe device, which eliminates complex remote storage stacks for edge workstations. Operations for remote storage access are implemented as hardware circuits inside the controller to eliminate CPU cycles. Parallelized and pipelined workflows are proposed for hardware circuits to accelerate remote storage access operations. Our evaluation presents that Leopard exhibits $1.09\times\sim6.04\times$ lower remote storage access latency than SOTA solutions for realistic workloads in edge intelligent workstations.

Index Terms—edge intelligent workstations, remote storage, hardware pass-through, queue concurrency, parallelized and pipelined workflows.

I. INTRODUCTION

In recent years, more and more edge intelligent workstations [11], [43] have been deployed on the data generation side to perform edge intelligence. They reserve limited computing (CPUs or GPUs) and storage resources to implement edge intelligent applications using edge data and massive empirical data. The large volume of empirical data is persistently stored in remote cloud storage servers (remote storage) [42] and needs to be loaded/stored via remote storage access on demand. Remote storage access is implemented using a software stack known as the remote storage access framework.

There are several remote storage access frameworks have been proposed, such as famous and widely used iSCSI [1] and NVMe Over Fabric (NVMe-oF) [28]. iSCSI is a single-queue framework for traditional hard disk drives (HDDs) [7]. It introduces unacceptable performance degradation and CPU overheads for modern multi-queue non-volatile memory express [30] solid state drives (NVMe SSDs) [13]–[16] due to its inefficient single-queue design. Specifically, iSCSI reduces the application-level throughput of NVMe SSDs by $>20\%$ [10]. Furthermore, it needs to consume 14 CPU cores for saturating a single modern multi-queue NVMe SSD [12]. NVMe-oF is a remote storage framework with a multi-queue design. It mitigates the performance degradation for remote access by leveraging multiple physical request queues in modern NVMe SSDs. NVMe-oF can be categorized into NVMe-over-TCP [31] (*nvof-tcp*) and NVMe-over-RDMA [29] (*nvof-rdma*) according to the transport layer. *nvof-tcp* is constructed based on TCP [8] network protocol. *nvof-rdma* is

constructed based on remote memory direct access (RDMA) [34] protocol and RDMA network card (RNIC) [23]. However, they are implemented as complex software stacks deployed on both Initiator and Target Hosts¹. Multiple CPU cores are used to manipulate multiple physical request queues in SSDs to saturate performance. Experiments in Section III show that *nvof-tcp* reduces remote access performance of NVMe SSDs to about 1/4 of local access when using a 100Gbps network card (NIC). And *nvof-rdma* has a performance degradation of about 10%~20% compared to local access [10].

Previous works [12], [26] have investigated the aforementioned disadvantages for traditional remote storage access frameworks. For example, an offloading method [26] is proposed to mitigate these disadvantages. Specifically, RNIC on the Target Host directly accesses NVMe SSDs in a peer-to-peer (P2P) manner, thereby slimming down the software stack on the Target Host. However, these works are proposed for cloud scenarios with sufficient CPU resources. Besides, they overlook multiple physical request queues in modern network devices, which can be used to improve concurrency for remote storage access. Figure 2 presents that these solutions still cannot avoid performance degradation for remote storage access in edge intelligent workstations when using modern high-performance NVMe SSDs. Therefore, we argue that existing remote storage access frameworks cannot fully leverage the queue concurrency in modern network devices and NVMe SSDs to implement high-performance remote storage access for edge intelligent workstations. The reasons are as follows: (i) The remote storage initiator (edge intelligent workstation) needs a complex software stack that integrates storage and network. (ii) The limited CPU resources in edge intelligent workstations cannot saturate multiple concurrent request queues in modern network devices and NVMe SSDs. (iii) The CPU still participates in remote storage access operations.

To this end, we propose Leopard, a hardware pass-through remote storage access framework using Field Programmable Gate Array (FPGA) [39]-based smart network cards (SmartNICs) [35]. It fully utilizes the queue concurrency in modern high-speed network devices and NVMe SSDs to implement lossless remote storage access for edge workstations. SmartNICs can independently perform tasks and optimize performance with low cost and low power consumption, such as [5], [20]–[22], [41]. The network functions in modern emerging SmartNICs also allow us to focus on investigating remote storage access workflows comprehensively without overly focusing on the network. First, Leopard customizes a native NVMe driver-compatible NVMe controller using SmartNIC's FPGA core to eliminate complex software stacks for Initiator Host. The Initiator Host only needs to submit remote storage requests without implementing remote-related operations. Second, Leopard constructs several FPGA-based accelerators that utilize multiple request queues in network devices and NVMe SSDs to accelerate remote storage access operations. Fi-

This work was funded by the National Key Research & Development Program of China (No. 2022YFB4502002), NSFC (No. 62402317 & 62032008), STCSM (No.24ZR1435500), and supported by the project from Wuxi Institute of Advanced Technology, a grant from State Key Laboratory of Avionics Integration and Aviation System-of-Systems Synthesis, and the Fund of Kunpeng&Ascend Center of Excellence, Shanghai Jiao Tong University. The corresponding authors are Bo Peng and Jianguo Yao.

¹In this paper, we refer to edge workstations that perform edge computing and initiate remote storage requests as the Initiator Host, and remote cloud storage servers that manage persistent storage devices as the Target Host.

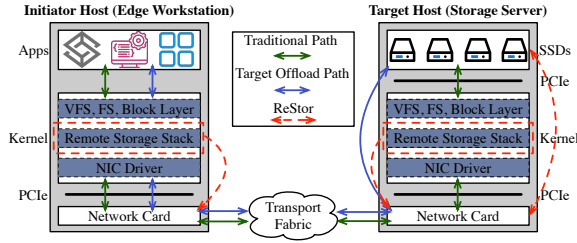


Fig. 1: Request/data path for remote storage access frameworks.

nally, Leopard accesses NVMe SSDs on remote cloud storage servers via SmartNIC's onboard peripheral component interconnect express (PCIe) slots. Leopard eliminates software stacks for Initiator Host and Target Host and provides queue-granularity remote storage access acceleration, thus optimizing remote storage access performance for edge workstations. The following are the main contributions:

- 1) We propose Leopard, a hardware pass-through remote storage access framework based on SmartNICs. It enables remote storage performance close to local access for edge intelligent workstations with limited CPU resources.
- 2) We propose several hardware accelerators for remote storage access, which construct queue-granularity parallelized and pipelined workflows to fully exploit multiple request queues in network devices and NVMe SSDs. They accelerate request and data processing for remote storage access.
- 3) We implement a Leopard prototype and evaluate remote access latency and throughput using micro-benchmarks and realistic workloads. Experiments for realistic workloads exhibit that the remote read/write latency of Leopard is consistent with local access, which is $1.09\times\sim6.04\times$ better than SOTA solutions.

II. PRELIMINARIES AND RELATED WORK

A. Preliminaries

Edge Workstation. Edge workstations [11], [17], [43] are deployed in several fields to implement edge intelligence. They are implemented in remote sensing [33], healthcare [9], [24], manufacturing [4], and agriculture [36], [38] scenarios to collect geospatial, image, and sensor data. They also perform edge intelligent applications using empirical data and collected data. However, edge workstations typically reserve limited computing (CPUs/GPUs) and storage resources required by applications. The large amount of empirical data is persistently stored in remote cloud storage servers. Edge workstations read/write empirical data from/to remote cloud storage servers via remote storage access (frameworks). Therefore, remote storage access in this scenario affects the performance of applications.

Remote Storage Access. Figure 1 depicts the breakdown of remote storage access paths for traditional and target offloading [26] frameworks. Remote storage requests are initiated by applications on the Initiator Host and transmitted to NVMe SSDs on the Target Host. Remote access paths of traditional frameworks (*nvof-tcp* and *nvof-rdma*) are the longest. Specifically, storage requests initiated by applications (Apps) need to go through software stacks (file system, block layer, remote storage stack, and driver) and NIC on the Initiator Host, as well as NIC and software stacks on the Target Host. The target offloading framework utilizes RNIC to manipulate (P2P) NVMe SSDs, eliminating software stacks and PCIe overheads on the Target Host. However, remote storage access operations still consume edge workstation's CPUs due to software stacks present on the Initiator Host. Besides, existing frameworks utilize multiple CPU cores to operate multiple physical request queues in NVMe SSDs, thereby maximizing SSDs' performance. Unfortunately, CPU resources in edge intelligent workstations are limited. Therefore, complex software stack hierarchy and CPU dependency prevent traditional and target offloading frameworks from implementing efficient

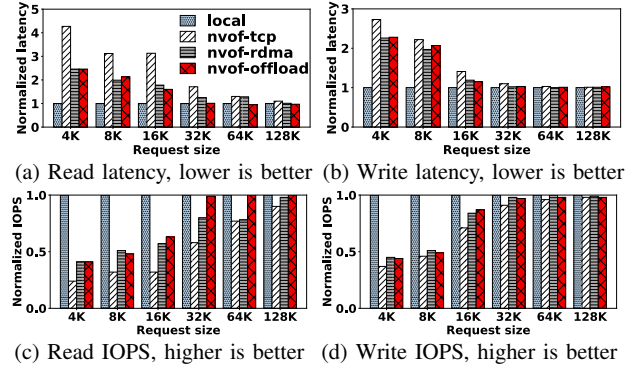


Fig. 2: Remote read/write latency and throughput for *nvof-tcp*, *nvof-rdma*, and *nvof-offload*. request size=4KB(K)~128KB(K), thread count=1, request depth=4, I/O engine=libaio.

remote storage access for edge intelligent workstations. Next, we provide an overview of existing remote storage access frameworks.

B. Related Work

Traditional Remote Storage Frameworks. iSCSI [1] is a traditional remote storage access framework designed for low-speed HDDs. It is unable to provide low-latency and high-throughput remote storage access as the development of modern network and storage devices. *NVMe-oF* [28] was proposed in 2014 and scaled to Ethernet, Fiber, and InfiniBand [32] in 2016. *NVMe-oF*'s multi-queue design achieves better latency and throughput performance for remote storage access compared to iSCSI. *nvof-tcp* and *nvof-rdma* are two classic *NVMe-oF* protocols. *nvof-tcp* introduces significant performance overhead compared to local access since the network layer adopts the TCP protocol, which is implemented by CPU software. *nvof-rdma* is proposed to mitigate the performance degradation for remote storage access using RDMA protocol and RNIC. However, the remote access performance of *nvof-rdma* in edge intelligent workstations is still 10%~20% worse than local access.

Remote Storage Framework Optimization. Several solutions [12], [19], [26] for optimizing existing remote storage access frameworks have been proposed. For example, *i10* [12] is proposed to optimize the performance of *nvof-tcp*. Its performance is similar to *nvof-rdma*. Highly optimized pipelines between CPU cores and remote storage access stacks are proposed to optimize bottlenecks at the boundary of the network stack and NVMe storage stack. Besides, NVME-OF TARGET OFFLOAD [26] is proposed to use RNIC's P2P ability for directly accessing NVMe SSDs, thus eliminating software stacks and PCIe overheads on the Target Host. ReFlex [19] designs and implements user-space storage and network frameworks to access remote Flash storage efficiently. However, the deployment and migration of user-space frameworks are complicated. They not only do not support POSIX interfaces but also are user-aware. Besides, these frameworks are designed for cloud scenarios with sufficient CPU resources. Their remote access performance is still worse than local access for edge workstations with limited CPUs.

III. PERFORMANCE CHARACTERISTICS

We conduct experiments to investigate performance bottlenecks and optimization opportunities for existing remote storage access frameworks in edge intelligent workstations. The software/hardware configuration of experiments is introduced in Section V. Intel SSD DC P4600 (P4600) [16] is used as the target disk.

Latency. Figures 2a and 2b present the remote read/write latency of existing remote storage access frameworks. Flexible I/O (FIO) [3] workloads with different request sizes (4KB~128KB) are used to collect evaluation data. The thread count and request depth of workloads are 1 and 4, and the I/O engine is libaio. *local* indicates

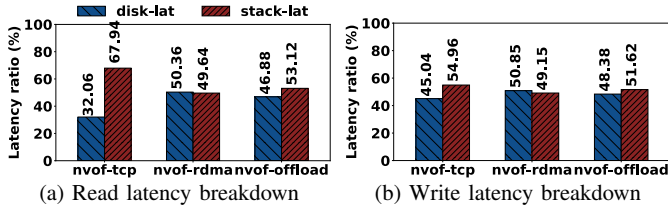


Fig. 3: Remote read/write latency breakdown for *nvof-tcp*, *nvof-rdma*, and *nvof-offload*.

TABLE I: The size of data in a single storage request for applications in edge intelligent workstations.

Data Type	Data Size	Data Type	Data Size
Sensor data [40]	<10KB	Log file	<100KB
Device status	<10KB	Image [18]	<100KB
Calculation result	<10KB	Configuration file	<100KB

accessing local SSDs, and *nvof-offload* indicates the target offloading framework proposed in [26]. Latency values are normalized to *local*. We make an observation from Figure 2a that the remote access constructed by existing remote storage access frameworks has higher read/write latency than *local*. Specifically, the remote read latency of *nvof-tcp* is $1.1 \times \sim 4.2 \times$ higher than the local read when the request size is $<128\text{KB}$. The remote read latency is still $1.63 \times$ and $1.52 \times$ higher than the local read on average, even using *nvof-rdma* and *nvof-offload* frameworks. Similarly, the remote write latency (Figure 2b) of *nvof-tcp*, *nvof-rdma*, and *nvof-offload* is higher than the local write when the request size is $<32\text{KB}$. **Therefore, existing remote storage access frameworks cannot provide low-latency remote storage access for edge intelligent workstations.**

Throughput. The number of I/O requests processed per second (IOPS) is the metric for storage throughput. FIO workloads in latency experiments are used to collect the remote read/write throughput for *nvof-tcp*, *nvof-rdma*, and *nvof-offload* frameworks. The remote read/write throughput exhibits a similar trend to the remote read/write latency, as shown in Figures 2c and 2d. The remote read/write IOPS of *nvof-tcp*, *nvof-rdma*, and *nvof-offload* frameworks is reduced to 1/4 and 1/3 of the local access when the request size is $<128\text{KB}/32\text{KB}$. **The throughput observation reveals that existing remote storage access frameworks in edge intelligent workstations cannot exploit the high throughput of modern high-speed NVMe SSDs.**

It is worth noting that the latency and throughput penalties introduced by existing frameworks decrease with increased read/write size. The insight is that NVMe SSDs handle small-size read/write faster than software remote storage access stacks, resulting in remote access stacks being the bottleneck for small-size remote read/write. On the contrary, SSDs require more time to handle large-size read/write, which leads to the overhead of remote access stacks being negligible as the read/write size increases. Therefore, existing remote storage access frameworks are more suitable for large-size read/write scenarios in edge intelligent workstations. However, **the small-size read/write is a common I/O characteristic for practical applications in edge intelligent workstations**, as shown in Table I.

Latency Breakdown. We breakdown the read/write latency of 8KB FIO workload for *nvof-tcp*, *nvof-rdma*, and *nvof-offload* frameworks to investigate the performance bottleneck. As shown in Figure 3, *disk-lat* represents the read/write latency of the SSD, and *stack-lat* represents the latency of remote storage access stacks. It is evident that *stack-lat* accounts for $>49\%$ of remote read/write latency in *nvof-tcp*, *nvof-rdma*, and *nvof-offload* frameworks, which results in SSD having higher remote read/write latency than local access. Next, optimization methods and insights in Leopard for mitigating these overheads are discussed.

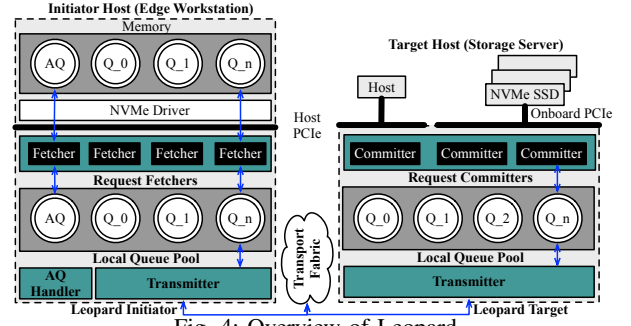


Fig. 4: Overview of Leopard.

IV. LEOPARD DESIGN

A. Leopard Overview

We propose a hardware pass-through framework called Leopard. It consists of two SmartNICs, as shown in Figure 4. We customize them as Leopard Initiator and Leopard Target and connect them to Initiator Host and Target Host via PCIe. **Request Fetchers, Local Queue Pool, Admin Queue Handler (AQ Handler), Transmitter, and Request Committers** are five novel accelerators. The processing of NVMe storage requests, including remote storage, is characterized by high density and high concurrency, as stated in the NVMe (Over Fabric) specification [28], [30]. FPGA is more suitable for concurrent tasks than traditional processors (such as x86 and ARM [2] CPUs). It also reserves the flexibility to customize hardware interfaces, including PCIe. Therefore, Leopard adopts FPGA-based SmartNICs instead of ARM-based SmartNICs [27].

Request Fetchers & Request Committers. Request Fetchers are implemented in Leopard Initiator. They parallelly fetch storage requests from Initiator Host memory using the direct memory access (DMA) engine of PCIe. It aims to minimize the fetching latency for remote storage requests as much as possible. Similarly, Request Committers are implemented in Leopard Target. They parallelly submit storage requests to NVMe SSDs using SmartNIC's onboard PCIe DMA engine. It aims to minimize the latency of remote storage requests reaching NVMe SSDs. The philosophy behind these accelerators is the potential parallelism between multiple NVMe request queues and multiple PCIe DMA channels. Specifically, multiple PCIe DMA channels in SmartNICs are used to parallelly fetch remote storage requests from multiple NVMe request queues in Initiator Host memory. They are also used to parallelly submit requests to multiple NVMe request queues in SSDs.

Local Queue Pool. Local Queue Pool is allocated in Leopard Initiator and Leopard Target to cache remote storage requests, aiming to construct multiple independent (queue-granularity) request processing pipelines. The Local Queue Pool in Leopard Initiator caches remote storage requests fetched from Initiator Host memory. And the Local Queue Pool in Leopard Target caches remote storage requests received from the Transmitter (Transport Fabric, Network). It consists of multiple isolated first-in-first-out (FIFO) buffers, corresponding to multiple NVMe request queues ($Q_0 \sim Q_n$) that Leopard Initiator exposes to Initiator Host. Leopard Initiator follows NVMe specification [30], thus each exposed NVMe request queue consists of a submission queue (SQ) and a completion queue (CQ), called SQ/CQ pair. Every two FIFOs are firmly bound to an SQ/CQ pair. One FIFO is used to cache pending requests submitted by the Initiator Host to SQ. Another FIFO is used to cache completion requests submitted by NVMe SSDs to CQ. The FIFO width is set to 512 bits for achieving compatibility with NVMe storage requests. The depth of FIFO is set to 1024 (1K) to construct request processing pipelines at queue granularity. Note that FIFOs are implemented using FPGA's on-chip memory that supports independent read/write. It aims to leverage parallelism in NVMe request queues and PCIe DMA channels.

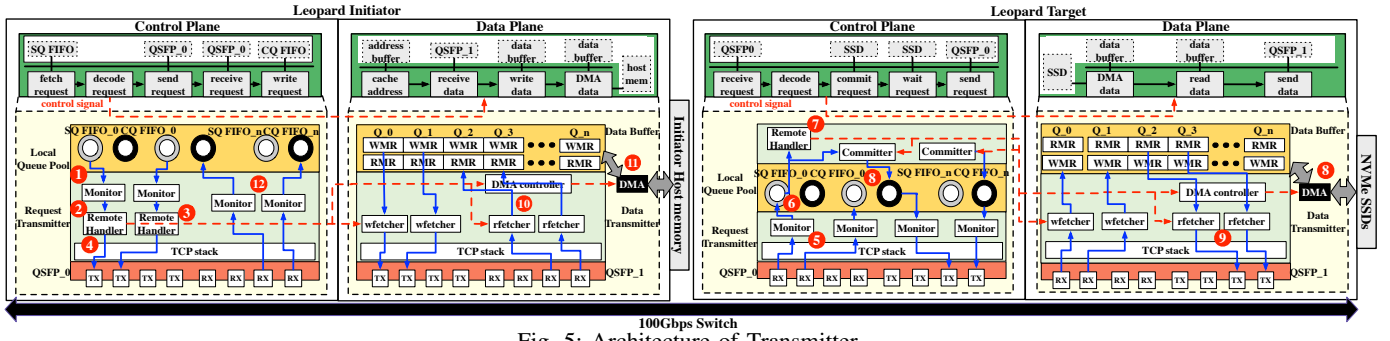


Fig. 5: Architecture of Transmitter.

AQ Handler. Initiator Host needs a dedicated software stack to implement remote storage access using existing frameworks. The software stack in edge workstations introduces performance overheads and CPU costs, as discussed in Sections III and II-A. AQ Handler disguises Leopard Initiator as a standard NVMe device compatible with the native NVMe driver, which eliminates remote storage access stacks for Initiator Host. In other words, AQ Handler is a customized NVMe controller. It allows Initiator Host's CPUs to simply submit remote storage requests to Leopard Initiator (via the native NVMe driver) for remote access without additional operations. The remote-access-related workflow is implemented by Leopard Initiator and Target. AQ Handler uses a workflow that follows the NVMe specification [30] to process NVMe admin requests submitted by the Initiator Host. NVMe admin requests are mainly used to manage and identify Leopard Initiator, such as creating and deleting NVMe request queues ($Q_0 \sim Q_n$). Therefore, admin requests submitted to the admin queue (AQ) are only cached to the corresponding FIFO in Leopard Initiator's Local Queue Pool, then processed by AQ Handler without being transmitted to Leopard Target.

Transmitter. Transmitter is the core accelerator for Leopard, which implements efficient hardware pass-through remote storage access. It not only accelerates storage and network stacks but also better utilizes multiple request queues in network devices and NVMe SSDs compared to existing solutions. Transmitter constructs a hardware pass-through queue-granularity parallelized and pipelined remote storage request processing flow between Leopard Initiator and Leopard Target, which eliminates CPU involvement. It mitigates software stacks and CPU overheads introduced by current remote storage access frameworks for Initiator and Target Hosts. Remote Handler and Monitor are its two hardware components. Remote Handler is used to implement the control plane for remote storage access (including remote storage requests and data). Monitor is used to poll FIFOs in Local Queue Pool or network receivers to fetch remote storage requests as fast as possible, thereby reducing latency. Next, we will discuss the detailed design for Request Fetchers, Request Committers, and Transmitter, which are closely related to remote storage access performance.

B. Design Details

Request Fetchers & Committers. The same number of Fetchers and Committers as NVMe request queues are implemented in Request Fetchers and Committers (Figure 4) to achieve queue-granularity fetching and submitting for remote storage requests. They exploit the parallelism of PCIe DMA channels in SmartNICs and NVMe request queues in SSDs by proposing independent hardware units (instead of multiple CPU cores). Fetcher and Committer consist of one Monitor and one DMA Controller, which are used to implement control and data planes for fetching and submitting remote storage requests.

Fetcher's workflow is as follows. **S1:** Monitor monitors the SQ of NVMe request queues in Initiator Host memory. It also monitors corresponding CQ FIFO in the Local Queue Pool of Leopard Initiator.

S2: As soon as the Monitor detects a new remote storage (completion) request in SQ (CQ FIFO), it sends address information and control signal to the DMA Controller. **S3:** DMA Controller then manipulates PCIe DMA to copy remote storage requests from Initiator Host memory to corresponding SQ FIFO located in the Local Queue Pool of Leopard Initiator. Completion requests are copied from CQ FIFO to CQ of the corresponding NVMe request queue located in the Initiator Host memory.

The workflow of Committer is similar to Fetcher. The difference is that it implements queue-granularity NVMe request interaction with NVMe SSDs via onboard PCIe DMA in Leopard Target. It eliminates software stacks, as well as PCIe and CPU overheads for remote storage access in Target Host. We equip each SQ/CQ pair with a Fetcher and a Committer to fully utilize the parallelism between NVMe request queues. The independent Fetcher and Committer are also adopted within each SQ/CQ pair to achieve parallelized transmission for the SQ/CQ pair's internal requests.

Transmitter. Software stacks and limited CPU resources are the bottleneck for small-size remote read/write in edge workstations, as discussed in Sections III and II-A. Transmitter is proposed to further mitigate software stacks and CPU overheads for Initiator Host and Target Host. It implements hardware pass-through processing for remote storage requests and data to replace software remote storage access stacks. It also leverages parallelism among multiple NVMe request queues, multiple network channels, and multiple PCIe DMA channels to construct parallelized and pipelined workflows. Figure 5 depicts the architecture of Transmitter, which proposes a highly parallelized and pipelined processing flow. It is divided into a control plane and a data plane. Four full-duplex network (QSFP_0) channels are used in the control plane to transmit remote storage requests parallelly. Another four full-duplex network (QSFP_1) channels are used in the data plane to transmit data triggered by requests.

We use remote read to demonstrate the Transmitter's workflow. **①:** In Leopard Initiator, independent Monitors poll corresponding SQ FIFO in the Local Queue Pool and notify the Remote Handler in the control plane if the SQ FIFO is not empty. **②:** The Remote Handler fetches remote storage requests from the non-empty SQ FIFO. Note that multiple Remote Handlers are deployed to implement queue-granularity acceleration. **③:** Remote Handler decodes the request's opcode field (OPC, byte 0). If OPC is the read operation, the Remote Handler decodes the target address and size of the data from the request. They are buffered into the corresponding DMA address buffer (in the DMA controller) in the data plane. The corresponding rfletcher in the data plane is also notified to poll the data receiving channel in QSFP_1. **④:** Then, requests are sent to Leopard Target via the hardware TCP stack and dedicated channel in QSFP_0. Leopard adopts TCP for compatibility with most edge scenarios.

Leopard Target further processes remote storage requests. Specifically, **⑤:** The multiple Monitors in Leopard Target's control plane poll four receiving channels in QSFP_0. **⑥:** Monitors dispatch received requests to corresponding SQ FIFO in the Local Queue Pool. **⑦:** The

TABLE II: Configuration of Initiator and Target Hosts.

Initiator Host (Edge Workstation)	AMD Ryzen 5 5600X 6-Core processor Mellanox ConnectX-5 [23] Western Digital (WD) HDD, 1TB DDR4 2666MHz, 16GB FPGA-based SmartNIC [35]
Target Host (Remote Cloud Storage Server)	Intel(R) Xeon(R) CPU E5-2680 processor Mellanox ConnectX-5 Western Digital (WD) HDD, 4TB DDR4 3200MHz, 64GB Intel SSD DC P4600 (P4600) [16], 2TB FPGA-based SmartNIC

TABLE III: Hardware cost & power of Leopard Initiator & Target.

	LUT	LUTRAM	FF	BRAM	Power (W)
Initiator	173719	10703	248981	536	16.67
	33.23%	6.64%	23.82%	54.47%	
Target	110721	10111	190547	386	14.19
	21.18%	6.27%	18.23%	39.23%	

Remote Handler in Leopard Target's control plane fetches requests from SQ FIFO and parses OPC and data address fields. If OPC is the read operation, the Remote Handler utilizes the source address and size of the data to inform the DMA controller in Leopard Target's data plane. Besides, the Committer is also notified to submit requests to the corresponding request queue in SSD via onboard PCIe and wait for SSD's completion request. ⑧: The data will be read by DMA (manipulated by the DMA controller) from SSD to the memory region (RMR) of the corresponding SQ FIFO in the data buffer. The SSD's completion request is written to CQ FIFO by Committer and transmitted to Leopard Initiator by Monitor via QSFP_0. ⑨: Hardware TCP stack in Leopard Target's data plane encapsulates data in RMR into TCP packets and sends them to Leopard Initiator via the dedicated channel in QSFP_1.

Leopard Initiator then processes data. ⑩: The refetcher in Leopard Initiator receives data from the receiving channel in QSFP_1 and writes it to the corresponding RMR in the data buffer. ⑪: The data in RMR will be transferred by DMA to the target address (cached in Step ③) in Initiator Host memory. ⑫: Finally, the Monitor in the Leopard Initiator's control plane receives the completion request from the Leopard Target and writes it to the corresponding CQ FIFO in the Leopard Initiator's Local Queue Pool. Note that the data buffer is constructed using FIFO for correctness, which allows data to be processed in order of request submission.

The workflow for remote write is similar to remote read, but the data transmission direction is from Initiator Host memory to SSDs. We propose optimizing steps ①~⑫ using queue-granularity parallelized and pipelined workflows, thereby accelerating the remote-access-related processing for storage requests and data.

V. EVALUATION

Hardware Setup. Two FPGA-based SmartNICs [35], one edge workstation (Initiator Host), and one cloud storage server (Target Host) are used to construct the testbed, as listed in Table II. The FPGA-based SmartNIC contains one Xilinx xczu19eg-ffvc1760-2-e FPGA core and several communication interfaces, such as PCIe and QSFP [25] (100Gbps). We implement the Leopard prototype on FPGA (Frequency 250 MHz) using Verilog. Two FPGA-based SmartNICs are implemented as Leopard Initiator and Leopard Target, respectively. They connect to the Initiator Host and Target Host via PCIe. PCIe is configured as Gen3×8 in our implementation. Leopard is also compatible with high version PCIe (Gen4 or Gen5). Besides, Target Host is equipped with a 2TB Intel SSD DC P4600 (P4600) to test remote access performance for Leopard and baselines. The Initiator Host and Target Host are connected to a 100Gbps switch via Mellanox ConnectX-5 (RDMA NIC), and P4600 is connected to

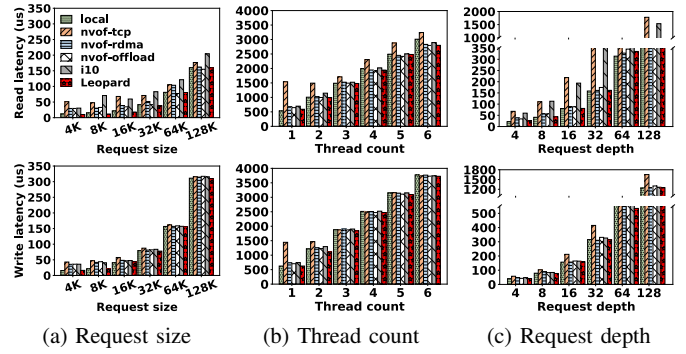


Fig. 6: Remote read and write latency comparison using FIO workloads, lower is better.

the Target Host via rack PCIe for testing baselines. The Initiator Host and Target Host are connected to the 100Gbps switch via Leopard, and P4600 is directly connected to Leopard Target via onboard PCIe for testing Leopard.

Software Setup & Baselines & Metrics. Ubuntu 20.04.4 LTS operating system with Linux kernel 5.10.73 is installed on the Initiator Host and Target Host. FIO (version 3.31) [3], RocksDB [37], and YCSB [6] are installed to generate evaluation workloads. All workloads are running on Initiator Host. We adopt the following baselines: 1) *local*: This is local access performance obtained by directly accessing local P4600 on the Initiator Host. 2) *nvof-tcp* [31]: Software remote storage access framework using TCP. 3) *nvof-rdma* [29]: Software remote storage access framework using RDMA. 4) *i10* [12]: Software optimized *nvof-tcp*. 5) *nvof-offload* [26]: *nvof-rdma* with target offload. We adopt remote read/write latency, throughput, and CPU cost to evaluate performance.

Hardware Cost & Power. The hardware cost and power consumption of Leopard are directly proportional to the number of parallel request queues (workflows). However, edge intelligent workstations typically adopt devices with low cost and low energy to minimize the total cost of ownership (TCO). Fortunately, we find that excessive queues (workflows) in Leopard result in unacceptable channel arbitration overhead due to limited channels in SmartNIC's PCIe DMA and network interfaces. The arbitration overhead will compromise remote storage access performance. Leopard adopts 32 queues (workflows), which is less than the request queues within modern NVMe SSDs, but evaluation shows that it is sufficient to saturate modern SSDs. If faster SSDs are used in the future, Leopard's design can scale at queue granularity using sufficient FPGA resources to maximize SSDs' remote access performance. Leopard Initiator and Target only consume 29.54%/21.23% of FPGA resources on average (Table III), which is approximately \$174.65/\$125.66 in hardware cost. The power consumption of Leopard Initiator and Leopard Target is 16.67 watts (W) and 14.19W, which is lower than implementing remote storage access using CPUs (>100W).

Latency and Throughput. We use remote read/write latency and throughput metrics to compare Leopard with baselines. FIO is installed on the Initiator Host. It generates sequential read/write workloads with different request sizes (4KB~128KB), different thread counts (1~6), and different request depths (4~128) to access SSD on the Target Host via Leopard and baseline frameworks. The I/O engine (ioengine parameter in FIO) is set to libaio. Existing remote storage access frameworks are unfriendly to small-size requests in edge workstations, as discussed in Section III. Therefore, the small-size workloads are adopted in experiments with different request sizes to present Leopard's improvements for edge workstations. Figure 6 presents the remote read/write latency. It is obvious that only Leopard enables remote read/write latency to be consistent with

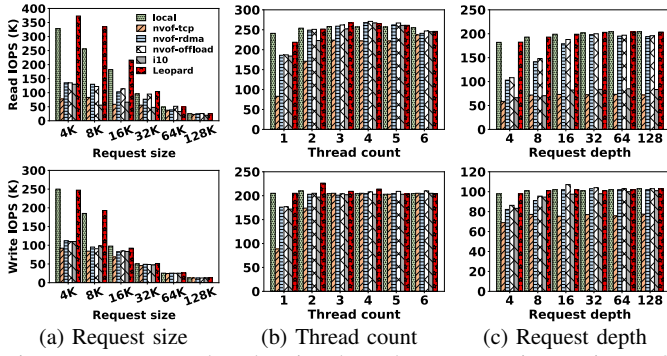


Fig. 7: Remote read and write throughput comparison using FIO workloads, higher is better.

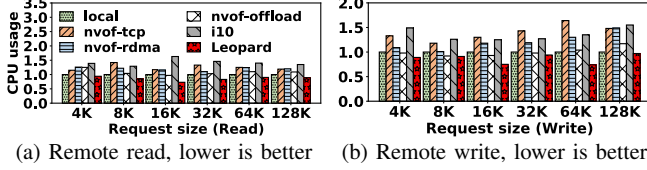


Fig. 8: CPU cost for remote read/write, normalized to *local*.

local read/write (*local*). Compared to baselines (*nvof-tcp*, *nvof-rdma*, *nvof-offload*, and *i10*), Leopard's remote read (write) latency for workloads with different parameters (request size, thread count, and request depth) is reduced by $2.59\times$, $1.75\times$, $1.66\times$, and $2.61\times$ ($1.49\times$, $1.34\times$, $1.36\times$, and $1.34\times$) on average. Leopard's remote read/write throughput is also better than baselines, achieving the same throughput as local read/write. The remote read (write) throughput of Leopard is $2.55\times$, $1.73\times$, $1.63\times$, and $2.60\times$ ($1.51\times$, $1.35\times$, $1.38\times$, and $1.36\times$) higher than baselines on average, as shown in Figure 7.

Leopard is more suitable for scenarios with small request sizes, according to Figures 6a and 7a. The response of SSD is fast in this scenario, causing existing complex remote storage access frameworks to become a bottleneck, as discussed in Section III. However, the small-size request is a typical remote storage access type for real applications in edge workstations (Table I). Leopard optimizes this bottleneck by (1) Customizing a highly parallelized and pipelined hardware pass-through workflow for fast remote storage access. (2) Integrating remote access tasks into FPGA-based SmartNIC to mitigate software layers and PCIe and CPU overheads for remote storage access. Therefore, Leopard is more suitable for edge workstations.

CPU Cost. Existing frameworks use multiple CPU cores to maximize the remote access performance for SSDs, as discussed in Section II-A. This not only introduces latency overhead but also incurs additional CPU cycles for edge intelligent workstations with limited CPU resources. We measure CPU usage in Initiator and Target Hosts for Leopard and baselines using FIO workloads with different request sizes (4KB~128KB). The queue depth and thread count are 16 and 8. The I/O engine is set to libaio. Figure 8 presents the experimental results, which are normalized to *local* values for intuitive comparison. The CPU usage of Leopard is close to local access. Compared to baselines (*nvof-tcp*, *nvof-rdma*, *nvof-offload*, and *i10*), Leopard's CPU usage for remote read (write) is reduced by $2.90\times$, $1.58\times$, $1.33\times$, and $2.22\times$ ($2.22\times$, $1.76\times$, $1.41\times$, and $1.83\times$).

Figure 9 also presents the CPU usage in kernel space for Leopard and baselines. Leopard's read (write) CPU usage in kernel space is $1.64\times$, $1.50\times$, $1.28\times$, and $1.80\times$ ($1.74\times$, $1.45\times$, $1.13\times$, and $1.69\times$) lower than baselines. The advantage of CPU cycles for Leopard comes from two factors. (1) Leopard Initiator is compatible with the native NVMe driver. It allows Initiator Host's CPUs to simply submit remote storage requests to Leopard Initiator for remote access without additional operations. Leopard Initiator enables edge workstations to access SSDs on remote cloud storage servers as if they were accessing

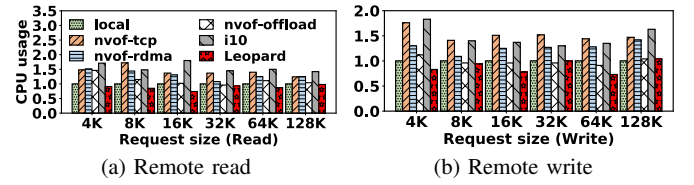


Fig. 9: CPU cost in kernel space for remote frameworks, normalized to *local*, lower is better.

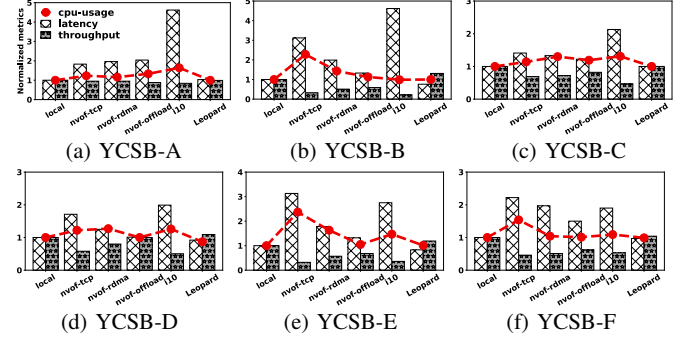


Fig. 10: Remote read/write comparison using YCSB workloads, all experimental values are normalized to *local*.

local SSDs. (2) The processing flow (including storage and network) for remote read/write is handled by Leopard Initiator and Leopard Target without CPU intervention. Therefore, Leopard is more suitable for edge intelligent workstations with limited CPUs.

Realistic Workloads. The latency, throughput, and CPU usage (cpu-usage) of remote storage access for Leopard and baselines are collected using realistic workloads to evaluate the benefits of Leopard in real-world scenarios. The YCSB [6] workloads based on RocksDB [37] are used to collect evaluation metrics. YCSB is a widely used evaluation tool for storage systems. It can generate standard and customized realistic workloads. Figure 10 collects the remote access latency, throughput, and CPU usage for Leopard and baselines using standard YCSB-A~YCSB-F workloads. YCSB-A~YCSB-F are six different realistic workloads. They have different request types (read-only, write-only, mixed, read-modify-write, etc), different request sizes, different read/write ratios, different thread counts, and different request depths. All experimental values are normalized to *local* for intuitive comparison. As shown in Figures 10a~10f, Leopard provides a remarkable optimization in remote storage access latency and CPU usage for YCSB-A~YCSB-F, achieving an optimization of $1.09\times\sim6.04\times$ and $1.03\times\sim2.34\times$ compared to existing remote storage access frameworks. It also exhibits a $1.04\times\sim6.02\times$ improvement for throughput. More importantly, Leopard provides the same latency, throughput, and CPU usage for remote storage access and local storage access. This experiment presents that Leopard effectively maintains remote storage access performance consistent with local access for realistic workloads in edge intelligent workstations.

VI. CONCLUSION

We present Leopard, an efficient hardware pass-through remote storage access framework based on SmartNIC for edge intelligent workstations. The end-to-end remote storage access workflow is formulated as multiple accelerators in SmartNIC's FPGA core. The AQ Handler helps Leopard adopt the native NVMe driver, thus eliminating the dedicated software stack. Request Fetchers, Request Committers, Local Queue Pool, and Transmitter construct a hardware pass-through remote storage access workflow using pipeline architecture. In the workflow, the concurrency of request queues, DMA channels, and network interfaces is fully utilized for fast remote access. The evaluation presents that Leopard provides lossless remote storage access for edge intelligence in edge intelligent workstations.

REFERENCES

- [1] S. Aiken, D. Grunwald, A. R. Pleszkun, and J. Willeke, "A performance analysis of the iscsi protocol," in *20th IEEE/11th NASA Goddard Conference on Mass Storage Systems and Technologies*, 2003.(MSST 2003). Proceedings. IEEE, 2003, pp. 123–134.
- [2] M. N. Asghar, "A review of arm processor architecture history, progress and applications," *Journal of Applied and Emerging Sciences*, vol. 10, no. 2, pp. pp–171, 2020.
- [3] J. Axboe, "Fio: Flexible i/o tester," <https://github.com/axboe/fio>, (Accessed 8 March 2024).
- [4] A. Bonello, E. Francalanza, and P. Refalo, "Smart and sustainable human-centred workstations for operators with disability in the age of industry 5.0: A systematic review," *Sustainability*, vol. 16, no. 1, p. 281, 2023.
- [5] M. Bonola, G. Belocchi, A. Tulumello, M. S. Brunella, G. Siracusano, G. Bianchi, and R. Bifulco, "Faster software packet processing on fpga nics with ebpf program warping," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 987–1004.
- [6] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.
- [7] W. Digital, "WD Gold Enterprise Class SATA HDD," <https://www.westerndigital.com/products/internal-drives/wd-gold-sata-hdd?sku=WD1005FBYZ>, (Accessed 10 November 2023).
- [8] B. A. Forouzan, *TCP/IP protocol suite*. McGraw-Hill Higher Education, 2002.
- [9] W. Gong, Z. Xu, L. Yuan, H. Zhou, S. Chen, Y. Ding, A. Nayak, and J. Liu, "Embracing self-powered wearables for intelligent healthcare data management," *IEEE Internet of Things Journal*, 2024.
- [10] Z. Guz, H. Li, A. Shayesteh, and V. Balakrishnan, "Performance characterization of nvme-over-fabrics storage disaggregation," *ACM Transactions on Storage (TOS)*, vol. 14, no. 4, pp. 1–18, 2018.
- [11] K. Hsu and H. Tseng, "Accelerating applications using edge tensor processing units," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.
- [12] J. Hwang et al., "Tcp~rdma: Cpu-efficient remote storage access with i10," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020, pp. 127–140.
- [13] Intel, "Intel Optane SSD 900P Series," <https://www.intel.com/content/www/us/en/products/sku/123626/intel-optane-ssd-900p-series-480gb-12-height-pcie-x4-20nm-3d-xpoint-specifications.html>, (Accessed 10 November 2023).
- [14] Intel, "Intel Optane SSD DC P4800X/P4801X series," <https://www.intel.com/content/www/us/en/products/docs/memory-storage/solid-state-drives/data-center-ssds/optane-ssd-dc-p4800x-p4801x-brief.html>, (Accessed 10 November 2023).
- [15] Intel, "Intel® optane™ dc ssd series," <https://www.intel.com/content/www/us/en/products/details/memory-storage/data-center-ssds/optane-dc-ssd-series.html>, (Accessed 10 November 2023).
- [16] Intel, "Intel® ssd dc p4600 series," <https://ark.intel.com/content/www/us/en/ark/products/series/96947/intel-ssd-dc-p4600-series.html>, (Accessed 10 November 2023).
- [17] A. Isosalo, J. Islam, H. Mustonen, E. Räämä, S. I. Inkinen, M. Brix, T. Kumar, J. Reponen, M. T. Nieminen, and E. Harjula, "Local edge computing for radiological image reconstruction and computer-assisted detection: A feasibility study," *Finnish Journal of eHealth and eWelfare*, vol. 15, no. 1, pp. 52–66, 2023.
- [18] V. Jeyakumar, K. R. Abirami, S. Saraswathi, R. S. Kumaran, and G. Marthi, "Secure medical image storage and retrieval for internet of medical imaging things using blockchain-enabled edge computing," in *Intelligent Edge Computing for Cyber Physical Applications*. Elsevier, 2023, pp. 85–110.
- [19] A. Klimovic, H. Litz, and C. Kozyrakis, "Reflex: Remote flash~local flash," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 345–359, 2017.
- [20] D. Kwon, J. Boo, D. Kim, and J. Kim, "Fvm: Fpga-assisted virtual device emulation for fast, scalable, and flexible storage virtualization," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 955–971.
- [21] J. Lin, T. Ji, X. Hao, H. Cha, Y. Le, X. Yu, and A. Akella, "Towards accelerating data intensive application's shuffle process using smartnics," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 2, pp. 1–23, 2023.
- [22] J. Liu, A. Dragojević, S. Flemming, A. Katsarakis, D. Korolija, I. Zablotchi, H.-c. Ng, A. Kalia, and M. Castro, "Honeycomb: ordered key-value store acceleration on an fpga-based smartnic," *IEEE Transactions on Computers*, 2023.
- [23] N. Mellanox, "NVIDIA Mellanox ConnectX-5 Ethernet Adapter Cards User Manual," https://img-en.fs.com/file/user_manual/connectx-5-ethernet-adapter-cards-user-manual.pdf, (Accessed 10 November 2023).
- [24] K. Mishra and S. Yadav, "Edge intelligence—a smart healthcare scenario in ambient-assisted living," in *Reconnoitering the Landscape of Edge Intelligence in Healthcare*. Apple Academic Press, 2024, pp. 157–171.
- [25] Q.-D. MSA, "Qsfp-dd hardware specification for qsfp double density 8x pluggable transceiver," *Revision*, vol. 1, pp. 1–49, 2019.
- [26] NVIDIA, "Nvme-of target offload," <https://enterprise-support.nvidia.com/s/article/howto-configure-nvme-over-fabrics--nvme-of--target-offload>, (Accessed 1 November 2023).
- [27] M. NVIDIA, "Nvidia mellanox bluefield smartnic for ethernet," <https://network.nvidia.com/files/doc-2020/pb-bluefield-smart-nic.pdf>, (Accessed 10 November 2023).
- [28] NVMEEXPRESS, "Nvm express over fabrics," <https://nvmexpress.org/wp-content/uploads/NVMe-over-Fabrics-1.1a-2021.07.12-Ratified.pdf>, (Accessed 1 November 2023).
- [29] NVMEEXPRESS, "Nvm express rdma transport specification," <https://nvmexpress.org/wp-content/uploads/NVMe-Express-RDMA-Transport-Specification-1.0b-2022.10.04-Ratified.pdf>, (Accessed 1 November 2023).
- [30] NVMEEXPRESS, "Nvm express specification," https://nvmexpress.org/wp-content/uploads/NVMe-Express-Base-Specification-2_0-2021.06.02-Ratified-5.pdf, (Accessed 1 November 2023).
- [31] NVMEEXPRESS, "Nvm express tcp transport specification," <https://nvmexpress.org/wp-content/uploads/NVMe-Express-TCP-Transport-Specification-1.0c-2022.10.03-Ratified.pdf>, (Accessed 1 November 2023).
- [32] G. F. Pfister, "An introduction to the infiniband architecture," *High performance mass storage and parallel I/O*, vol. 42, no. 617-632, p. 102, 2001.
- [33] D. A. Quattrochi, N. S.-N. Lam, H.-L. Qiu, and W. Zhao, "Image characterization and modeling system (icams): a geographic information system for the characterization and modeling of multiscale remote sensing data," in *Scale in remote sensing and GIS*. Routledge, 2023, pp. 295–307.
- [34] R. Recio, B. Metzler, P. Culley, J. Hilland, and D. Garcia, "A remote direct memory access protocol specification," *Tech. Rep.*, 2007.
- [35] Resnics, "Stargate-f1000-sn," <http://www.resnics.com/product/netcard>, (Accessed 1 November 2023).
- [36] A. Sanaeifar, M. L. Guindo, A. Bakhshipour, H. Fazayeli, X. Li, and C. Yang, "Advancing precision agriculture: The potential of deep learning for cereal plant head detection," *Computers and Electronics in Agriculture*, vol. 209, p. 107875, 2023.
- [37] F. D. E. Team, "Rocksdb: A persistent key-value store for flash and ram storage," <https://github.com/facebook/rocksdb>, (Accessed 20 March 2024).
- [38] H. Thai, K. Le, and N. L. Nguyen, "Towards sustainable agriculture: A lightweight hybrid model and cloud-based collection of datasets for efficient leaf disease detection," *Future Generation Computer Systems*, vol. 148, pp. 488–500, 2023.
- [39] S. M. Trimberger, *Field-programmable gate array technology*. Springer Science & Business Media, 2012.
- [40] T. Wang, Y. Liang, X. Shen, X. Zheng, A. Mahmood, and Q. Z. Sheng, "Edge computing and sensor-cloud: Overview, solutions, and directions," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–37, 2023.
- [41] Z. Wang, H. Huang, J. Zhang, F. Wu, and G. Alonso, "Fpganic: An fpga-based versatile 100gb smartnic for gpus," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 967–986.
- [42] W. Zhang, Z. Shi, Z. Liao, Y. Li, Y. Du, Y. Wu, F. Wang, and D. Feng, "Graph3po: A temporal graph data processing method for latency qos guarantee in object cloud storage system," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–16.
- [43] Z. Zou, Y. Kim, F. Imani, H. Alimohamadi, R. Cammarota, and M. Imani, "Scalable edge-based hyperdimensional learning system with brain-like neural adaptation," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.